

14
2.50

(Let's dry out all subsequences
with the given constraint)

↙
pick the one
with the
maximum
sum.

↘
(Recursion)





$f(\underline{m})$

```
if (ind == 0) return a[ind];  
if (ind < 0) return 0;
```

→ $pick = \underline{a[ind]} + f(\underline{ind - 2}) ;$

1. Ind ✓

2. Do Stuffs
on that Index

3 Return best.

①



2.50

if (ind == 0) return a[ind];

3 ✓ Return best.

if (ind < 0) return 0;

→ pick = a[ind] + f(ind - 2);

→ not pick = 0 + f(ind - 1);
return max(pick, not pick);

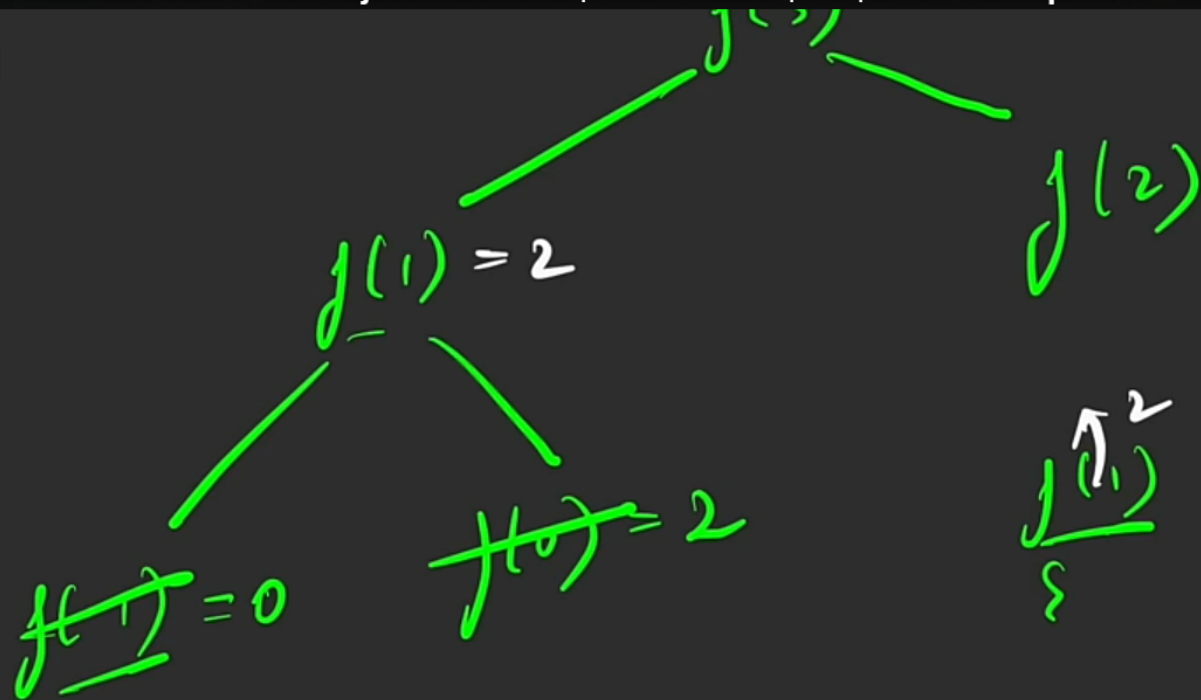
}

TC → $O(2^n)$

(L6)



2.50



$$\frac{j(1)}{2}$$

$$\begin{aligned} \text{prev} &= a[1] + j(0) \\ \text{curr} &= 0 + j(1) \end{aligned}$$



2.50

Tabulation $dp[n] \rightarrow 0$ $dp[0] = a[0]; \quad \checkmark$ int neg = 0; $\checkmark$ $\text{for}(i = 1; i < n; i++)$
 $\{$ $\text{take} = a[i]; \text{ if } (i > 1) \text{ take} += dp[i-2];$ $\text{notake} = 0 + dp[i-1];$ $dp[i] = \max(\text{take}, \text{notake});$

0 of 53



2.50

Tabulation $dp[n] \rightarrow 0$ $dp[0] = a[0]; \quad \checkmark$ $int \text{ prev2} = 0; \quad \checkmark$ $for (i = 1; i < n; i++)$

{

 $take = a[i]; \quad if (i > 1) \text{ take} += \text{prev2};$ $non\text{take} = 0 + \text{prev};$ $dp[i] = \max(\text{take}, \text{non}\text{take});$

}



2.50

```
int neg = 0;
```

```
for (i = 1; i < n; i++)
```

```
{
```

```
take = a[i]; if (i > 1) take += prev2dp[i-2];
```

```
nonTake = 0 + dp[i-1] prev;
```

```
dp[i] = max(take, nonTake);
```

```
prev2 = prev, prev = cur;
```

Tabulation $\rightarrow$ Space Optimization

$TC \rightarrow O(n)$ $SC \rightarrow O(n)$
--



DP 5. Maximum Sum of Non-Adjacent Elements | House Robber | 1-D | DP on Subsequences



Practice Guided Paths Interview Prep Challenges New Knowledge Centre New Community New

Problem Submissions Solution

Correct Answer

Report

Submitted on Jan 11, 2022, 11:20:12 PM

Penalty ? Score ?

100%

0

Runtime 325ms
Language C++ (g++ 5.4)

Previous Submissions

Submitted On	Status	Score	Penalty	Run
23 hours ago	Correct Answer	0	100%	415r
23 hours ago	Correct Answer	0	100%	410r
23 hours ago	Correct Answer	0	100%	325r
23 hours ago	Time Limit Exceeded	0	100%	-

```

1  int f(int ind, vector<int> &nums, vector<int> &dp) {
2      if(ind == 0) return nums[ind];
3      if(ind < 0) return 0;
4
5      if(dp[ind] != -1) return dp[ind];
6
7      int pick = nums[ind] + f(ind - 2, nums, dp);
8      int notPick = 0 + f(ind - 1, nums, dp);
9
10     return dp[ind] = max(pick, notPick);
11 }
12 int maximumNonAdjacentSum(vector<int> &nums){
13     int n = nums.size();
14     int prev = nums[0];
15     int prev2 = 0;
16     for(int i = 1; i < n; i++) {
17         int take = nums[i];
18         if(i > 1) take += prev2;
19
20         int notTake = 0 + prev;
21
22         int curi = max(take, notTake);
23         prev2 = prev;
24         prev = curi;
25     }
26     return prev;
27 }

```



Rate this problem

Previous

Next

Show Hint

Last saved on 11:31:08 PM

▶



30:43 / 32:22

Coding implementation >

